DOCUMENTATION
HOOKS DOCUMENTATION

Presentado por:

Isabella Garces Acosta
Diego Steven Pinzon Yossa
Juan Camilo Rosero Satisteban
Sharick Yelixa Torres Monroy
Laura Sofia Vargas Rodriguez

Profesora:
Liliana Marcela Olarte Mesa

Universidad Nacional de Colombia
Facultad de ingeniería
Departamento de Ingeniería de sistemas e industrial

2025

CONTENT

USE-ITINERARIES - CUSTOM REACT HOOK FOR ITINERARY DATA MANAGEMENT

This custom React hook (useItineraries) provides a streamlined interface for fetching, managing, and paginating user-specific itinerary data from the backend API. It encapsulates complex state management for itinerary collections, including loading states, error handling, and pagination logic with a clean, reusable API. The hook accepts a required userId parameter and an optional initialLimit for pagination control, defaulting to 16 items per page. Internally, it uses React's useState and useCallback hooks to manage the itinerary array, loading status, error messages, and pagination metadata. The core fetchItineraries function constructs API requests with proper pagination parameters (skip and limit), handles response parsing, and updates all related state variables. An automatic initial fetch is triggered via useEffect when the hook mounts or when dependencies change, while the exposed fetchItineraries method allows manual refresh and pagination navigation from consuming components. This hook demonstrates sophisticated data fetching patterns with proper error boundaries, TypeScript typing for the ItineraryLean model, and integration with the application's environment configuration for API base URL resolution. It serves as a critical data layer abstraction that simplifies itinerary listing across multiple components while maintaining consistent loading states and error handling.

USE-MOBILE - RESPONSIVE DESIGN DETECTION HOOK

This utility React hook (useIsMobile) provides a responsive design solution for detecting mobile viewport sizes with a configurable breakpoint threshold (default 768px). The implementation leverages the Window API's matchMedia method to create a media query listener that efficiently responds to viewport changes without continuous resize event polling. The hook initializes with an undefined state to handle server-side rendering scenarios in Next.js applications, then immediately evaluates the current viewport width on client-side hydration. It establishes an event listener for viewport changes that updates the isMobile state reactively when the browser crosses the defined breakpoint threshold. The cleanup function ensures proper removal of the event listener to prevent memory leaks when components unmount. This hook provides a performance-optimized alternative to CSS media queries in JavaScript contexts, enabling conditional rendering, dynamic styling, or component behavior adjustments based on device classification. Its boolean return value (coerced with double negation to handle initial undefined state) offers a simple, intuitive API for components to adapt their presentation or functionality across different screen sizes, supporting responsive design patterns throughout the application.